

微机系统与接口——

第三章 8086微处理器及其系统

东南大学信息科学与工程学院



王东明



1

微处理器

2

存储器及寻址

3

指令系统

逻辑和移位指令

- ◆ 逻辑运算指令
- ◆ 移位指令
- ◆ 循环移位指令

逻辑和移位指令分类

逻辑运算

- AND
- OR
- XOR
- NOT
- TEST

移位指令

- SAL
- SAR
- SHL
- SHR

循环移位指令

- ROL
- ROR
- RCL
- RCR



4

所有指令都是把操作数当做**二进制的位串**操作。



逻辑运算

AND 目, 源

置0: AND AL, 0FH

OR 目, 源

置1: OR AL, 0FH

XOR 目, 源

取反: XOR AL, 0FH

NOT 源

逻辑非: 所有位取反

TEST 目, 源

与运算, 不保留结果,
影响标志, 用于跳转。



逻辑运算

注意

- 逻辑运算指令是一组位操作指令，它们可以对字或字节按位执行逻辑操作，因此，源操作数经常是一个位串。
- 以上五条指令除NOT不影响标志位外，其它四条指令执行后，CF和OF置0，AF无定义，SF、ZF和PF根据运算结果设置。
- 测试指令TEST与AND指令的区别仅在于TEST不改变目操作数，其后常紧跟条件转移指令以实现判断分支。



对AL寄存器第7、5、2位置0，其它位不变。

AND **AL**, 01011011B

对AL寄存器第6、4、0位置1，其它位不变。

OR **AL**, 01010001B

对AL寄存器第7位取反，其它位不变。

XOR **AL**, 10000000B

AL=80H，测试AL寄存器的第7位，ZF标志？

TEST **AL**, 10000000B

思考这三条指令
执行后的结果。

XOR **AX**, **BX**
XOR **BX**, **AX**
XOR **AX**, **BX**



逻辑和移位指令

- ◆ 逻辑运算指令
- ◆ 移位指令
- ◆ 循环移位指令

移位指令

算术右移

算术右移是对带符号数进行右移，在移位过程中必须保持符号不变。

逻辑移位

逻辑移位是对无符号数移位，总是用0来填补已空出的位置。

- SAR: 算术右移。
- SHL: 逻辑左移, **SAL=SHL**。
- SHR: 逻辑右移。

指令格式 (以SAR为例) :

SAR 目, 1

SAR 目, CL



移位指令SHL

SHL CF  0

CF AL=6AH



0 1 1 0 1 0 1 0

CF

SHL AL, 1



0 1 1 0 1 0 1 0

0

CF

SHL AL, 1



1 1 0 1 0 1 0 0

0



移位指令SHR

SHR

0



CF

AL=6AH

CF

0 1 1 0 1 0 1 0



SHR AL, 1

0

0 1 1 0 1 0 1 0

CF



SHR AL, 1

0

0 0 1 1 0 1 0 1

CF



移位指令SAR

SAR



CF

符号位

AL=EAH

CF

1 1 1 0 1 0 1 0



SAR AL, 1

1

1 1 1 0 1 0 1 0

CF



SAR AL, 1

1

1 1 1 1 0 1 0 1

CF



EAH的符号位为1，移位过程中保持符号位不变。

移位指令SAR

符号位		AL=6AH	CF
		0 1 1 0 1 0 1 0	
SAR AL, 1	0	0 1 1 0 1 0 1 0	
SAR AL, 1	0	0 0 1 1 0 1 0 1	



6AH的符号位为0，移位过程中保持符号位不变。



移位指令

注意

- 允许的操作数类型：通用寄存器、存储器。
- 对标志位的影响：**移出位送至CF**，按照移位后的结果置/复位SF、ZF、PF。
- 移位位数超过1位，移位位数放到CL。 **SAR AL, 2** ❌



逻辑和移位指令

- ◆ 逻辑运算指令
- ◆ 移位指令
- ◆ 循环移位指令

循环移位指令

循环移位

循环移位指令是将操作数首尾相接进行移位，它分为不带进位和带进位循环移位。

带进位循环移位

带进位的循环移位相当于9位的循环移位。

- ROL: 不带进位的循环左移
- ROR: 不带进位的循环右移
- RCL: 带进位的循环左移
- RCR: 带进位的循环右移

指令格式（以ROL为例）：

ROL 目, 1

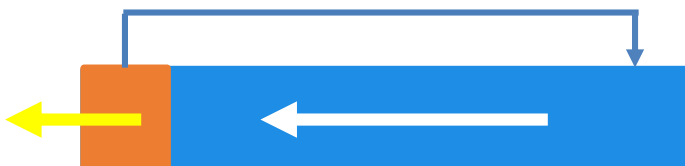
ROL 目, CL



循环移位指令ROL

ROL

CF



CF

AL=6AH



0 1 1 0 1 0 1 0

CF

ROL AL, 1



0 1 1 0 1 0 1 0

CF

ROL AL, 1



1 1 0 1 0 1 0 0



逻辑和移位指令小结

1 AND、TEST、OR、XOR根据运算结果改变SF、ZF、PF。

2 NOT指令形式上只有一个操作数，不影响任何标志位。

3 SAL、SAR、SHL、SHR指令的使用方法相同，对标志位的影响也相同：移出位给CF；根据结果改变SF、ZF、PF。



逻辑和移位指令小结

4

ROL、ROR、RCL、RCR的使用方法相同，对标志位的影响相同：移出位给CF。

5

使用SHR或SAR每移1位，可视为分别将无符号数和带符号数除以2；使用SHL/SAL每移1位，可视为将数值乘以2。



串操作指令

- ◆ 串处理指令
- ◆ 串重复前缀
- ◆ 设置方向标志

编程中常用到数组、矢量的复制、比较和搜索。

如何用汇编指令实现？会不会很复杂？



问题



要解决问题

- 连续的存储单元可看做是矢量、数组；
- 数组首地址、数组长度；
- 数组的索引，以及从首地址到尾、还是从尾到首；
- 重复操作（高级语言用for循环）。

串操作的概念

概念

串

地址连续的字节或字存储单元，通常存放同类数据。

串操作指令

对串元素实现功能性动作、存储等操作。

重复前缀

加在串操作指令前，根据条件判断是否重复串操作。

串操作方向

串操作可从准备工作从低地址到高地址。



串操作指令类似高级语言中的向量、数组操作，可有效加快数据处理速度，且指令简洁。



串操作指令的特点

隐含寻址

默认源串地址DS: SI, 目标串地址固定在ES: DI。串长度放在CX。因此, 必须预先设定源串、目标串地址、CX的值。

指针修改

标志寄存器中DF决定指针修改方向: DF = 0, 指针加; DF = 1, 指针减。修改量由串类型决定, 字节 ± 1 , 字 ± 2 。

计数器

为加快串操作, 加重复前缀时, 固定使用CX计录并修改重复次数, 再根据CX值及标志位ZF判断是否重复进行串操作。



对比高级语言, 是不是与for循环处理矢量和数组的思路类似?



串操作指令分类

串处理指令

- **MOVSB/MOVSW**
- **STOSB/STOSW**
- **LODSB/LODSW**
- **CMPSB/CMPSW**
- **SCASB/SCASW**

串重复前缀

- **REP**
- **REPE/REPZ**
- **REPNE/REPNZ**

设置方向标志

- **CLD**
- **STD**



串传送指令

- **MOVSB、MOVSW**：字节传送、字传送。执行操作：**[DS:SI] → [ES:DI]**，将源串的值传送到相应的目标串单元中，同时修改指针SI、DI。
- **DF的设置**：CLD，DF清零，STD，DF置1。
- 重复前缀**REP**：串操作指令执行后， $CX = CX - 1$ ，判断CX是否为0，不是0，继续执行串操作指令；否则执行下一条指令。

注意

- 修改指针： $SI = SI \pm 1$ (MOVSB) 或 ± 2 (MOVSW)， $DI = SI \pm 1$ (MOVSB) 或 ± 2 (MOVSW)。
- REP不能单独使用，必须配合串操作指令。格式：**REP MOVSB** 或 **REP MOVSW**



已设定 $ES=DS$ 。字串 ARRAY1，有16个字，它的尾部与ARRAY2的首部有部分交叠。如何把ARRAY1复制到以ARRAY2开始的32个字节中。

MOV CX, 10H • 串长度

LEA SI, ARRAY1

ADD SI, 1EH • 串地址

LEA DI, ARRAY2

ADD DI, 1EH

STD • 串方向

REP MOVSW



串比较指令

- **CMPSB、CMPSW**：用串源中DS:SI所指的串元素值减去目标串中ES:DI所指的串元素值，结果反映在标志位上；自动修改SI、DI。
- 重复前缀REPZ/REPE：串操作指令执行后 $CX = CX - 1$ ，如果 $CX \neq 0$ 且结果相等（ZF=1），则继续执行该串操作指令；否则顺序执行下一条指令。
- 格式：**REPE CMPSB** 或 **REPE CMPSW**



REPZ和REPE完全等价，哪个容易记忆用哪个。



字节串 STR1 和
STR2, 有20个字节。
比较它们是否相等。

```
MOV CX, 20
LEA SI, STR1
LEA DI, STR2
CLD
REPE CMPSB
JNE DIFF
SAME: .....
```



串检索指令

- **SCASB、SCASW**：用AL（字节串）或AX（字串）的值减去目标串中ES:DI所指的串元素值，结果反映在标志位上；然后自动修改DI。
- 重复前缀REPNE/REPZ：串操作指令执行后 $CX = CX - 1$ ，如果 $CX \neq 0$ 且结果不相等（ZF=0），则继续执行该串操作指令；否则顺序执行下一条指令。
- 格式：**REPNE SCASB** 或 **REPNE SCASW**



REPNE和REPZ完全等价，哪个容易记忆用哪个。



字节串STR2有30个字节，检索STR2中
有无字符'\$'。

```
MOV CX, 30
LEA DI, STR2
MOV AL, '$'
CLD
REPNE SCASB
JE FND
NOFND: .....
```



串装入和串存储指令

- **LODSB、LODSW**：将源串中DS:SI所指的串元素复制到AL（字节串）或AX（字串）中；然后自动修改SI。
- **STOSB、STOSW**：将AL（字节串）或AX（字串）的值复制到目标串中ES:DI所指的单元处；然后自动修改DI。

注意

- **LODSB、LODSW**实际上是目标串固定为累加器的串传送指令，一般不使用重复前缀！
- **STOSB、STOSW**实际上是源串固定为累加器的串传送指令，该指令可加重复前缀REP。



写出LODSB的等价指令。

```
MOV AL, [SI]  
INC SI
```

写出STOSB的等价指令。

```
MOV ES:[DI], AL  
INC DI
```

把从ES段中A000H开始的
2KB内存单元清零。

```
MOV DI, A000H  
XOR AX, AX  
MOV CX, 1024  
CLD  
REP STOSW
```



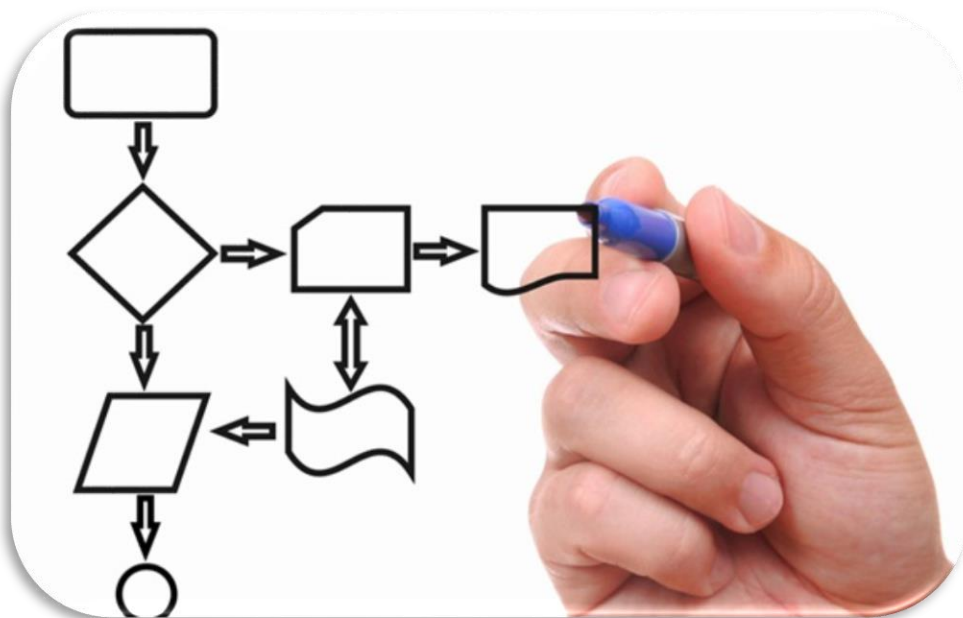


程序控制指令

- ◆ 无条件转移
- ◆ 条件转移
- ◆ 循环控制
- ◆ 中断



没有分支和循环，
程序员的世界是怎
么样的？



程序控制指令分类

无条件转移

- **JMP**
- **CALL**
- **RET**

条件转移

- **JA/JB/JAE/JBE**
- **JZ/JNZ/JE/JNE**
- **JG/JGE/JL/JLE**
- **JO/JNO/JP/JNP**
- **JC/JNC/JNS/JS**

循环控制

- **LOOP**
- **LOOPE/Z**
- **LOOPNE/Z**
- **JCXZ**

中断

- **INT**
- **INTO**
- **IRET**



程序控制指令特点

程序转移类指令可改变IP或CS、IP的内容，从而控制指令的执行顺序，实现指令转移、程序调用等功能。

一般都只有一个操作数，该操作数给出目标地址—新的IP（或CS:IP）值。指令对当前IP（或CS:IP）进行强制性修改，并执行相关操作。



程序控制指令

- ◆ 无条件转移
- ◆ 条件转移
- ◆ 循环控制
- ◆ 中断

无条件转移：JMP

- **JMP 目标操作数**：目标操作数可为立即数、寄存器、存储器。
- 根据目标操作数，直接或间接找到目标指令的地址。

分类

- 段内直接转移：分**短转移**和**近转移**。短转移用**单字节**、近转移用**双字节的有符号数位移**，表示目标地址与当前IP差值。
- 段内间接转移：目标地址放在寄存器或存储器中的两个字节。
- 段间直接转移：目标地址为4字节立即数，可用**远标号**定义。
- 段间间接转移：目标地址存放在存储器的连续4个字节中。



举例：直接转移（常用）

LABEL1: MOV xxxxxx

.....

JMP SHORT LABEL1

段内短转移

LABEL2: MOV xxxxxx

.....

JMP NEAR PTR LABEL1

段内近转移

LABEL1: MOV xxxxxx

.....

JMP FAR PTR LABEL1

段间远转移



举例：间接转移

段内转移仅改变IP

JMP BX
JMP [BX]

段内间接转移

段间转移改变CS: IP

JMP DWORD PTR [BX]

段间间接转移



间接转移的目标地址放在寄存器或存储器中。
JMP [BX]不用指定类型，默认段内间接转移。



问题

汇编语言中如何 实现子程序调用？



要解决问题

- 子程序的调用实质上是一种强制改变正常指令指针顺序的过程，调用之前的正常IP值称为“断点”。
- 三个要素：**断点保护、转移、返回**。

无条件转移：CALL/RET

- **CALL 目标操作数**：子程序调用指令，**将当前的IP或CS：IP压栈**，根据目标操作数修改IP或CS：IP，完成转移。
- **RET/RETF**：返回指令，用栈中数据修改IP/CS：IP完成转移。

- 汇编中的过程（子程序）有近过程和远过程两种。在调用近过程时，仅将IP压栈。调用远过程时，将CS：IP压栈。
- RET用于近过程的返回，仅修改IP，RETF用于远过程的返回，修改CS：IP。

注意



近过程：处于当前代码段的过程，其它代码段的过程称为远过程；如何确定近/远过程，后面讲伪指令解决。



CALL的寻址

- CALL指令不能实现短转移，除此之外，**CALL实现转移的方法与JMP指令相同。**
- CALL指令与JMP指令的区别在与CALL除了完成转移，还会把**当前的断点保护到栈。**

1

保护断点

近调用：当前IP压栈

远调用：当前CS、IP压栈

2

转移

近调用：获得子程序偏移地址给IP

远调用：获得子程序的段、偏移地址给CS: IP



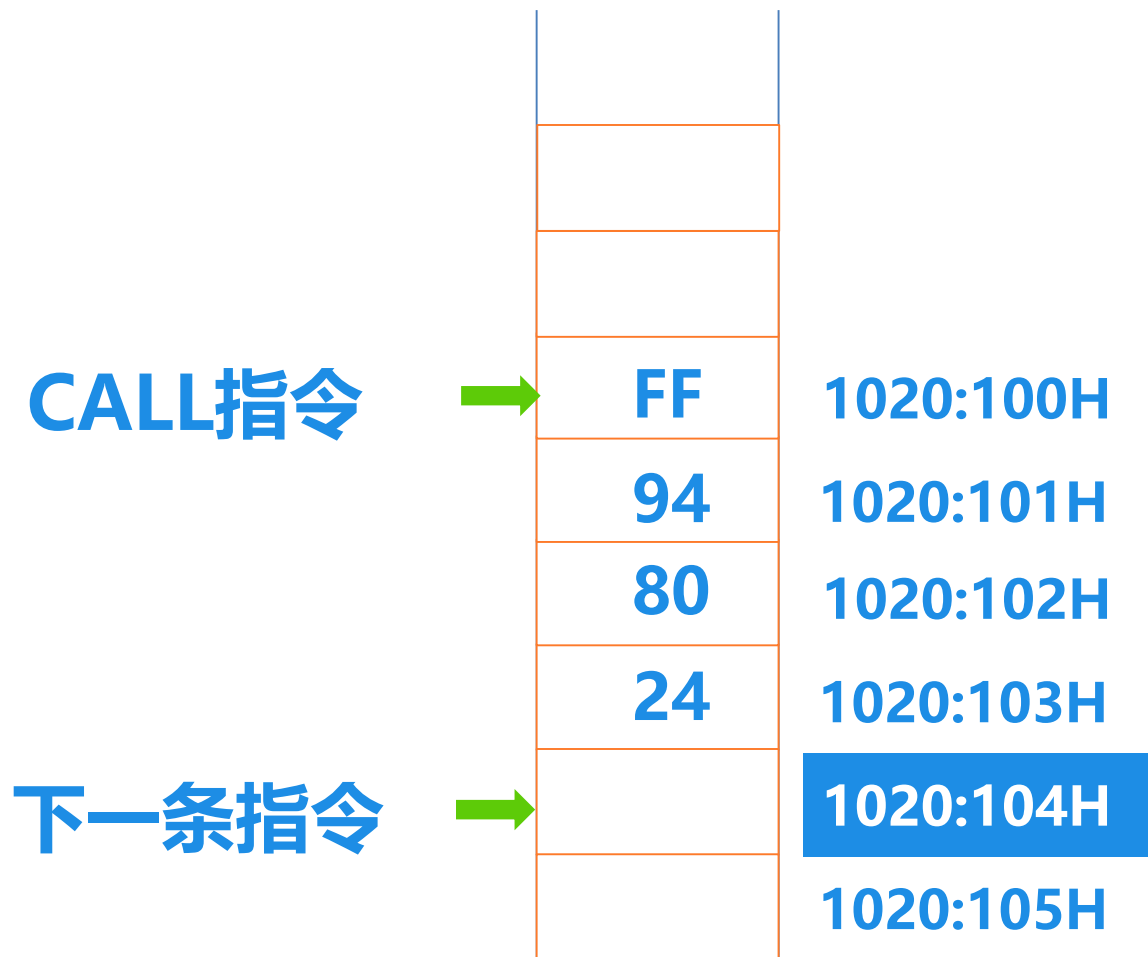
记不住压栈顺序？**CS在高地址，当然先压栈，因为压栈时SP指针是减的。** CALL涉及压栈，注意栈段及SP指针的变化。



举例：CALL的间接段间调用



CALL DWORD PTR [SI+2480H]



主程序代码段



CALL DWORD PTR [SI+2480H]

① CS压栈

② IP压栈

CS

10 20

IP

01 04

4FC

4FD

4FE

4FF

500

501

执行前

SP



SS段



CALL DWORD PTR [SI+2480H]

SI=1000H

DS: 3480H

57

13

DS: 3482H

68

24

DS段

IP

CS



CALL DWORD PTR [SI+2480H]

子程序入口

2468:1357H

RETF

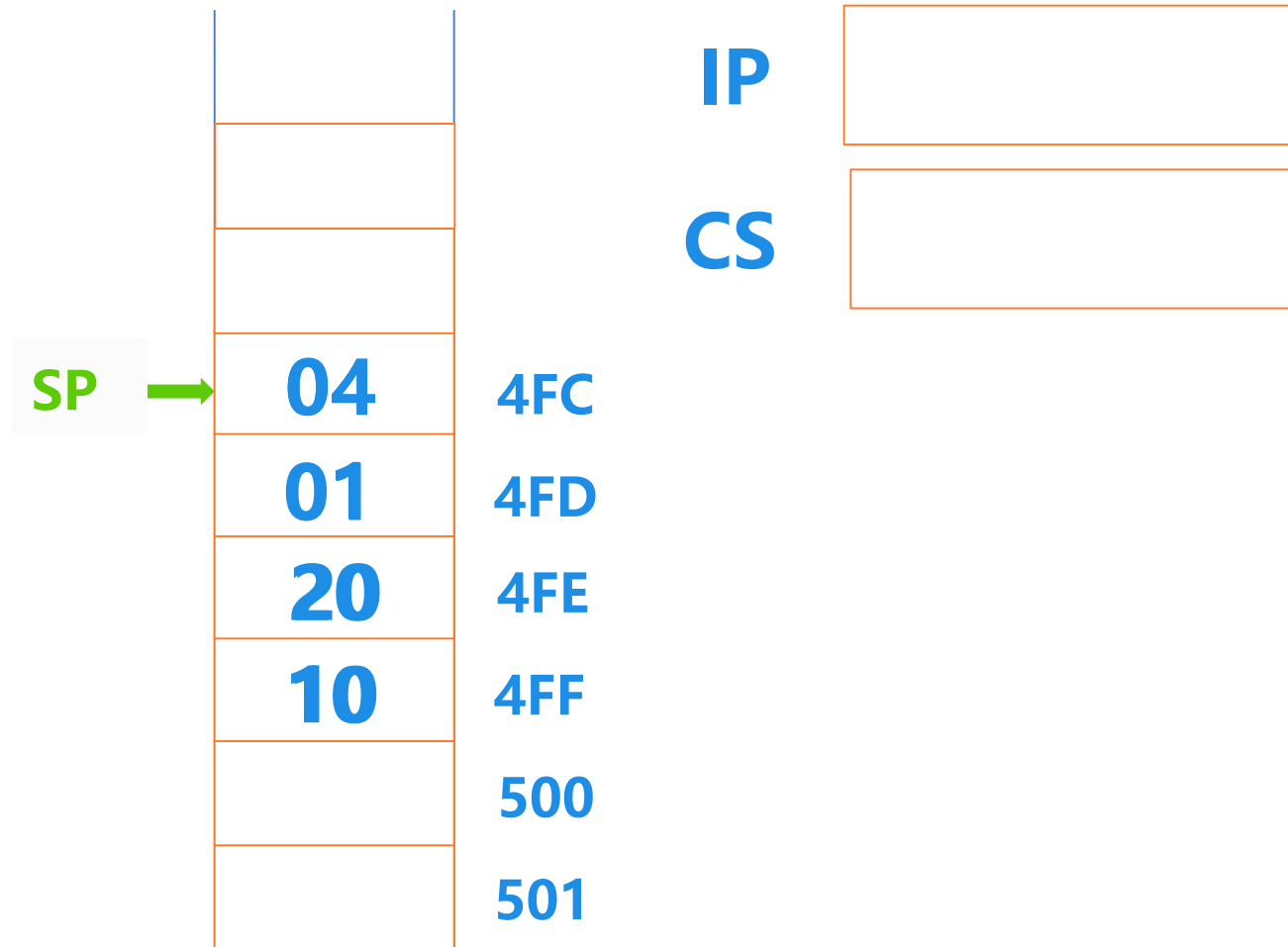


CB

子程序代码段



RETF指令执行示意图



举例

CS: 2000H **MOV** AX, 2012H
CS: 2003H **MOV** CX, 200CH
CS: 2006H **PUSH** CX
CS: 2007H **CALL** 4000H
CS: 200AH **ADD** AX, BX
CS: 200CH **ADD** AX, DX
CS: 200EH **HLT**

CS: 4000H **MOV** BX, 200AH
CS: 4003H **POP** DX
CS: 2006H **RET**

SS段

200A

200C

DX=200AH

IP=200CH

AX=2012H+200AH



程序控制指令

- ◆ 无条件转移
- ◆ 条件转移
- ◆ 循环控制
- ◆ 中断

问题

汇编语言中如何实现分支？



要解决问题

- 分支是代码中一个常用的结构，通常根据条件实现指令的执行顺序。
- 要素：**条件**、**转移**。
- 条件的形成：**通常**由指令**CMP**、**TEST**形成。

条件转移指令特点

1

条件转移指令：满足指令中要求的条件，则产生转移。格式：
指令 标号

2

根据上一条指令执行后CPU设置的状态标志（C、Z、S、O、P）做测试条件，决定是否转移。

3

所有条件转移指令都是短转移指令，目标地址在指令地址的-128至+127字节范围之内。

段内直接短转移

4

对于无符号数和有符号数分别使用不同的两组条件转移指令。



条件转移指令分类

对无符号数

- JA/JNBE
- JAE/JNB
- JB/JNAE
- JBE/JNA

Above/Below

结合CMP

对有符号数

- JG/JNLE
- JL/JNLE

Great/Less

结合CMP

单标志

- JE/JZ
- JNE/JNZ

Equal/CF

结合CMP或TEST

位条件转移

- JO
- JNO
- JP/JPE
- JNP/JPO
- JS
- JNS

OF、PF、SF

结合CMP

这里的 / 表示两个指令完全等价



以16进制的形式显示BX寄存器中的值。假设
BX=1234H，要求显示为 '1234H' 。

题目

思路

- 共4位16进制数，可从高位开始每次显示1位，需要4次，每次把BX循环左移4比特，取出低4比特。
- 0~F如何转换为相应的ASCII码？ASCII码的 '0' 与0相差30H， 'A' 与A相差37H。



	MOV BX, 1234H	
	MOV CH, 4	; CH循环计数器, 4个16进制
ROT:	MOV CL, 4	; CL移位计数器, 每次移4比特
	ROL BX, CL	; 最高4位移到低4位
	MOV AL, BL	
	AND AL, 0FH	; 取出最低4比特
	ADD AL, 30H	; 转换为ASCII码
	CMP AL, 39H	; 是0~9吗?
	JBE DISP	; 是则显示
	ADD AL, 7	; 否则是A~F, 加7, 转为ASCII
DISP:	MOV DL, AL	; 显示
	MOV AH, 2	
	INT 21H	
	DEC CH	
	JNZ ROT	; 未显示完? 下一位
	MOV DL, 48H	; 显示ASCII码 'H'
	MOV AH, 2	
	INT 21H	



```
MOV AL, 80H
CMP AL, 80H
JE    EQUAL
MOV DL, 0H
JMP LABEL
```

EQUAL:

```
MOV DL, 80H
```

LABEL:

```
MOV AL, 80H
TEST AL, 80H
JE    EQUAL
MOV DL, 0H
JMP LABEL
```

EQUAL:

```
MOV DL, 80H
```

LABEL:

两段程序执行后 DL分别等于？



让别人更容易读懂你的代码，右边代码最好用 JZ 或者 JNZ 更容易让人理解。

程序控制指令

- ◆ 无条件转移
- ◆ 条件转移
- ◆ 循环控制
- ◆ 中断

问题

汇编语言中如何实现循环？



循环指令

- 循环控制指令在循环代码中确定是否要继续循环。
- 循环次数置于**CX**中。
- 转移寻址方式：段内直接**短转移**。

循环指令

- **LOOP 标号**: 执行动作, $CX = CX - 1$, 若 CX 不为零, 跳转至标号, 否则结束循环。
- **LOOPZ/LOOPE 标号**: 执行动作, $CX = CX - 1$, 若 CX 不为零, 且 $ZF = 1$ (相等), 则跳转至标号, 否则结束循环。配合影响 ZF 的指令。
- **LOOPNZ/LOOPNE 标号**: 执行动作, $CX = CX - 1$, 若 CX 不为零, 且 $ZF = 0$ (不相等), 则跳转至标号, 否则结束循环。配合影响 ZF 的指令。
- **JCXZ 标号**: $CX = 0$, 则转移。它不修改 CX , 只根据 CX 的值转移。



要求将串长度为100字节的两串中各对应元素相加，所得新串写入目标串中。源串偏移地址为STR1，目标串偏移地址为STR2。ES=DS。

```
MOV CX, 100
LEA SI, STR1
LEA DI, STR2
CLD
AGN: LODSB
      ADD AL, [DI]
      STOSB
      LOOP AGN
```



程序控制指令

- ◆ 无条件转移
- ◆ 条件转移
- ◆ 循环控制
- ◆ 中断

中断指令

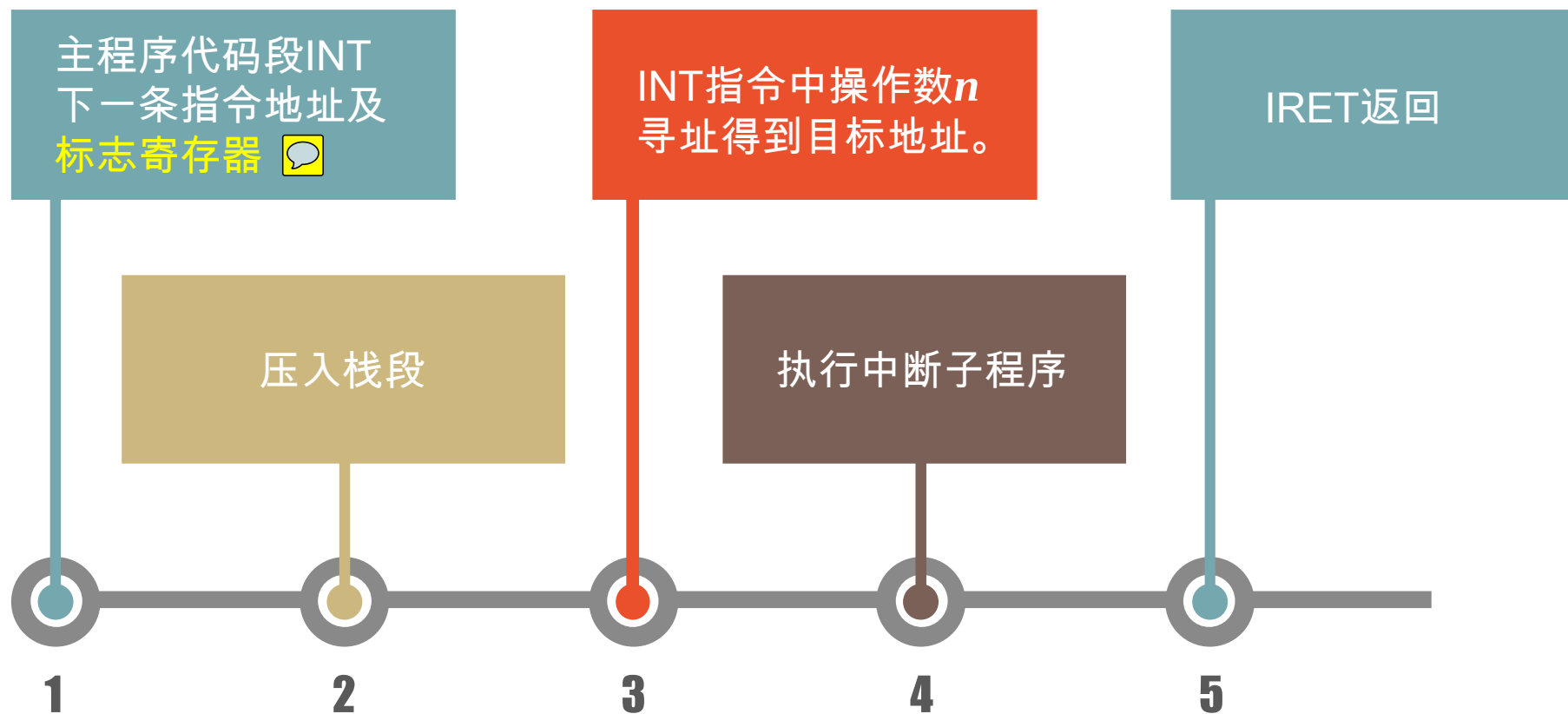
- 8086 CPU允许一条中断指令来产生中断过程，这种中断叫内部中断，或叫**软中断**。
- 软中断通常用于完成系统功能调用，例如DOS和BIOS调用。软中断类似**CALL**的段间间接调用。

注意

- 指令格式：**INT n**，其中 $n=0\sim 255$ 。💬
- 返回指令：**IRET**。
- 溢出中断：**INTO**， $OF=1$ ，则**执行类型4**的中断； $OF=0$ ，不做任何操作。



中断调用过程



INT指令的压栈顺序：标志寄存器、CS、IP。
IRET顺序刚好是反过程。

问题

INT指令如何只根据一个
序号 n 就能找到中断服务
子程序的地址?





对比 CALL DWORD PTR [SI+2480H]

中断向量

- 中断类型 n 的中断服务子程序入口地址放在**段地址为0**，**偏移地址为 $n*4$ 的连续4个字节中**。这4个字节存储单元也称为**中断向量**。
- 256个中断类型，需要1024个存储单元，即1KB，这1KB的存储单元称为**中断向量表**。
- 8086中断向量表在内存的最开始的1KB中。它的每4个字节为一个整体。因此，知道序号，就可以得到偏移地址： $4*n$ 。



INT 1FH指令的中断服务子程序放在1000H. 03FFH开始的存储空间里。画出右图中，该指令对应的中断向量的物理地址及内容。

$$1FH * 4 = 111\ 11\ 00B = 7CH$$

7CH

FF

7DH

03

7EH

00

7FH

10



程序控制指令小结

- 1 控制转移指令可实现远转移、近转移、短转移。
- 2 条件转移指令、循环控制指令只能实现短转移；
- 3 对无符号数和带符号数的大小的比较，应采用不同的条件转移指令，相应根据不同条件进行判断。
- 4 所有控制转移指令均不影响标志位。



CPU控制指令

处理器控制指令

- **CLC**: CF清零
- **STC**: CF置1
- **CMC**: CF取反

- **CLD**: DF清零
- **STD**: DF置1

- **STI**: IF置1, 开放外部的INTR中断
- **CLI**: IF清零, 禁止外部的INTR中断。



回答本章第三个问题

3

8086的指令分几类？主要指令的用法？

- 6类指令：数据传递、算术运算、逻辑和移位、串操作、程序控制、CPU控制。
- 掌握课件中所涉及的所有指令的用法，要求能够阅读相应的代码。

回答



检测点



1

判断对错：SHL AL, 4
INT CL



作业

作业 • HOMEWORK



习题： 3.36, 3.49,
3.50, 3.74
3.78, 3.82, 3.83

谢谢听讲

授课
教师

王东明

移动通信国家重点实验室



Tel: 13915982595



wangdm@seu.edu.cn



无线谷1-205

